LPI

Tema 103 — Comandos GNU y Unix

El tema que más pesa en el examen

Peso total: 26 de 90 — la sección más representativa del examen LPIC-1 101-500 v5.0. Cubre desde el shell bash y quoting hasta edición con vi, pasando por filtros, streams, procesos y expresiones regulares.

DISTRIBUCIÓN DE PESOS (W)

103.1	103.2	103.3	103.4	103.5	103.6	103.7	103.8	Σ
4	2	4	4	4	2	3	3	26

LPI

Recorrido del Tema 103

Cada subtema incluye comandos, trampas frecuentes y ejemplos copiables. Peso total = 26.

103.1 Trabajar en la línea de comandos — w = 4
bash, quoting, variables

103.2 Procesar textos con filtros — w = 2
cat, cut, sort, uniq, wc

103.3 Gestión de archivos básica — w = 4
cp, find, tar, dd, globs

103.4 Redireccionamiento de streams — w = 4
FD 0/1/2, >, >>, 2>&1, tee, xargs

103.5 Crear, monitorizar y matar procesos — w = 4
jobs, kill, ps, top, tmux

103.6 Modificar prioridades — w = 2
nice, renice, rango -20..19

103.7 Buscar con expresiones regulares — w = 3
BRE/ERE, grep, sed

103.8 Editar archivos con vi — w = 3
modos, 10 teclas, EDITOR

LPI

Trabajar en la línea de comandos

bash, variables de entorno y reglas de quoting

COMANDO	FUNCIÓN	EJEMPLO
echo	Imprime texto	echo $HOME
env	Lista variables exportadas	env | grep PATH
export	Crea/edita variable env	export EDITOR=vim
set	Lista TODAS las variables	set | grep ^PATH
unset	Elimina variable	unset EDITOR
type	Tipo de comando (alias/builtin/file)	type ll
which	Ruta del ejecutable	which bash
man	Manual del comando	man bash
uname	Info del kernel	uname -a
history	Histórico de comandos	history | tail

Pista de examen: type diferencia entre alias/builtin/executable; which sólo busca en PATH.

Quoting — 3 reglas

1. Comillas simples 'literal'

NO expande nada. echo '$USER' → $USER

2. Comillas dobles "expanden"

Expande variables y $(). echo "$USER" → alice

3. Sustitución de comando

$(cmd) reemplaza al backtick `cmd` (más legible y anidable)

LPI

Procesar textos con filtros

Encadenar comandos con |: cada salida es la entrada del siguiente




```
$ cat /etc/passwd | cut -d: -f7 | sort | uniq -c | sort -rn | head -5
     42 /bin/bash
     18 /usr/sbin/nologin
      3 /bin/sh
      1 /bin/false
```

⚠ TRAMPA DE EXAMEN `uniq` SOLO agrupa líneas IDÉNTICAS ADYACENTES. Sin `sort` previo, las duplicadas dispersas no se eliminan.

LPI

Gestión de archivos con comandos básicos

cp, find, tar, dd y patrones glob

Comandos clave

`cp -a`	—	copia preservando atributos
`mv`	—	mueve / renombra
`rm -rf`	—	borra recursivamente (¡cuidado!)
`find`	—	búsqueda por nombre/tipo/permisos
`tar`	—	archivar y comprimir
`dd`	—	copia a bajo nivel (if=/of=)

Globs más usados

`*`	—	cualquier cadena
`?`	—	un carácter
`[abc]`	—	uno de los listados
`[!abc]`	—	ninguno de los listados
`[a-z]`	—	rango

RECETA 1 — FIND + CP

```
# find + cp: backup de todos los .conf
$
 find /etc
-name
'*.conf'
-exec
 cp {} /backup/ \;
```

RECETA 2 — TAR (CREAR / EXTRAER / LISTAR)

```
# tar: crear y extraer (¡el orden de las flags importa!)
$
 tar
czf
 backup.tgz carpeta/
# crear gzip
$
 tar
xzf
 backup.tgz
# extraer gzip
$
 tar
tf
```

Pista: en tar, la flag f (file) debe ir JUSTO antes del nombre del archivo; czf = c(reate)+z(gzip)+f(file).

Redireccionamiento de streams

Cada proceso tiene 3 file descriptors (FD) abiertos por defecto



SINTAXIS	QUÉ HACE	EJEMPLO
`>`	Redirige stdout a archivo (trunca)	`ls > listado.txt`
`>>`	Añade stdout al final del archivo	`date >> log.txt`
`2>`	Redirige stderr	`cmd 2> errores.log`
`2>&1`	Combina stderr con stdout	`cmd > todo.log 2>&1`
`<`	Redirige stdin desde archivo	`wc -l < /etc/passwd`
`tee archivo`	Copia stdout a archivo Y a pantalla	`cmd | tee log.txt`
`xargs`	Construye argumentos desde stdin	`find . -name '*.log' | xargs rm`

```
$ find /var/log -name '*.log' 2>/dev/null | xargs grep ERROR 2>&1 | tee errores.txt
```

Crear, monitorizar y matar procesos

jobs, kill, ps, top, tmux y las señales que caen en el examen

terminal Comandos de control

`ps aux`	lista todos los procesos (BSD syntax)
`ps -ef`	equivalente POSIX
`jobs`	procesos en segundo plano del shell actual
`fg %1 / bg %1`	traer a primer plano / enviar al fondo
`top`	monitor interactivo en tiempo real
`tmux`	multiplexor de terminal, sesiones persistentes
`kill PID`	envía SIGTERM (15) por defecto

bolt Tabla de señales clave

SEÑAL	NÚM.	USO TÍPICO
SIGHUP	1	Colgar / recargar configuración
SIGINT	2	Ctrl+C — interrupción
SIGKILL	9	Muerte forzosa (NO atrapable)
SIGTERM	15	Terminación elegante (POR DEFECTO)
SIGCONT	18	Reanudar proceso detenido
SIGSTOP	19	Detener (NO atrapable)

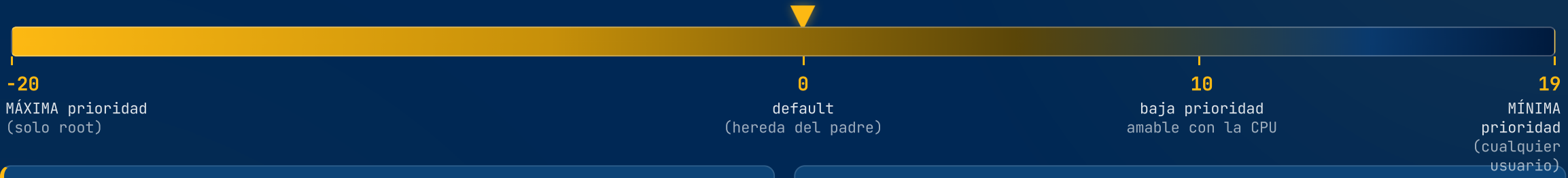
```
$ sleep 100 &
[1] 12345
$ jobs
[1]+  Running   sleep 100 &
$ kill %1        # envía SIGTERM al job 1
$ kill -9 12345  # SIGKILL al PID 12345
```

Trampa: kill %1 = job del shell; kill 1 = PID 1 (init). ¡Confundirlos es pregunta clásica!

Modificar prioridades de procesos

El rango nice va de −20 (mayor prioridad) a +19 (menor prioridad)

ESCALA NICE · -20 → +19



play_circle nice — arrancar con prioridad

```
$ nice -n 10 tar czf backup.tgz carpeta/
# el proceso arranca con prioridad 10 (más bajo que el
default 0)
```

tune renice — cambiar en marcha

```
$ renice -n 5 -p 12345
# ajusta la prioridad del PID 12345 a 5
$ renice -n -5 -p 12345   # requiere root
```

Trampa: valores NEGATIVOS (mayor prioridad) requieren root. Usuarios normales sólo pueden SUBIR el nice (bajar prioridad).

Búsqueda con expresiones regulares

BRE (básicas) vs ERE (extendidas), grep y sed

compare_arrows BRE vs ERE — diferencias

OPERADOR	BRE	ERE
a+ (1 o más)	a\+	a+
a? (0 o 1)	a\?	a?
a|b (alternancia)	a|b	a|b
(abc) (grupo)	\(abc\)	(abc)
{n,m} (rangos)	a\{2,4\}	a{2,4}

ERE elimina las barras invertidas — más legible.

search Tres modos de grep

`grep texto`	BRE por defecto
`grep -E texto`	ERE (igual que egrep)
`grep -F texto`	texto literal, sin regex
`grep -v`	invierte el match
`grep -i`	case insensitive
`grep -c`	solo cuenta coincidencias

EJEMPLO GREP

```
$ grep -E '^[a-z]+@example\.com$' emails.txt
# lista correos que empiezan por minúsculas y terminan en
@example.com
```

EJEMPLO SED (SUSTITUCIÓN)

```
$ sed 's/foo/bar/g' archivo.txt        # imprime el resultado
$ sed -i 's/foo/bar/g' archivo.txt     # sobrescribe el archivo
$ sed -i.bak 's/foo/bar/g' archivo.txt # guarda backup antes
```

Trampa: sed -i sobrescribe SIN backup. Usa sed -i.bak si dudas.

103.8 · W=3

Editar archivos con vi

Modos, atajos que SIEMPRE preguntan y variable EDITOR

MODO 1

MODO COMANDO

Por defecto al abrir. Navega y ejecuta acciones. Teclas como `x`, `dd`, `yy`, `p`, `u`.

MODO 2

MODO INSERCIÓN

Se entra con `i/a/o`. Permite escribir texto. Se sale con `ESC`.

MODO 3

MODO LÍNEA (Ex)

Se entra con `:`. Permite `:w :q :wq :q! :%s/foo/bar/g`.

TECLA	ACCIÓN	TRAMPA
`i`	Insertar antes del cursor	sale con ESC
`a`	Insertar después del cursor	sale con ESC
`o`	Abrir línea nueva DEBAJO	sale con ESC
`x`	Borrar carácter bajo cursor	sin buffer
`dd`	Borrar línea entera	va al buffer (pegable con p)
`yy`	Copiar (yank) línea	duplica con p
`p`	Pegar después del cursor	buffer de línea
`u`	Deshacer última acción	repite para más niveles
`:w`	Guardar sin salir	sin modificar nombre
`:q!`	Salir SIN guardar	descarta cambios

VARIABLE EDITOR

```
$ export EDITOR=vim      # vim es compatible con vi
$ export VISUAL=vim      # algunos programas leen VISUAL primero
# crontab -e, visudo, sudoers usan $EDITOR
```

LPI

Comandos que caen — y su trampa

Cada comando con el error que más confunde en el examen LPIC-1.

#	COMANDO	TRAMPA
1	`grep -E vs grep -F`	-E interpreta regex; -F trata literal
2	`find -exec \; vs +`	+ agrupa argv (más rápido)
3	`tar czf archivo`	la 'f' debe ir JUSTO antes del nombre
4	`dd bs=4M`	NO expande globs (*, ?)
5	`cp -a vs cp -r`	-a preserva atributos y enlaces
6	`mv (sin -i)`	sobrescribe sin preguntar
7	`rm -rf ./-archivo`	usa -- antes de nombres con guion
8	`ln -s destino nombre`	destino PRIMERO, nombre después
9	`kill -9 vs kill -15`	-9 NO permite cleanup del proceso
10	`kill %1 vs kill 1`	% = job del shell; número = PID

#	COMANDO	TRAMPA
11	`ps aux vs ps -ef`	BSD vs POSIX — ambos válidos
12	`top -b -n 1`	modo batch (no interactivo)
13	`nice -n -5`	valor negativo REQUIERE root
14	`renice -p PID`	-p es PID; -g es grupo de procesos
15	`sort -n vs sort -h`	-n numérico; -h humano (1K, 2M)
16	`cut -d ':' -f1`	el delimitador es UN solo carácter
17	`wc -l`	cuenta \n, no líneas reales
18	`uniq sin sort`	SOLO agrupa adyacentes idénticos
19	`sed -i`	sobrescribe SIN backup (usa -i.bak)
20	`:wq vs :x`	:x solo guarda si hubo cambios

Laboratorio /tmp/lpi103

Copia, pega y ejecuta. Cubre filtros, find, tar, pipes y ps.

1

```
# Preparar directorio de trabajo

$ mkdir -p /tmp/lpi103 && cd /tmp/lpi103
```

2

```
# Crear archivo y mostrar su contenido

$ echo 'hola mundo LPIC' > archivo.txt && cat archivo.txt
```

3

```
# Transformar a mayúsculas y redirigir

$ cat archivo.txt | tr a-z A-Z > MAYUS.txt && cat MAYUS.txt
```

4

```
# Buscar y contar líneas de cada .txt

$ find . -type f -name '*.txt' -exec wc -l {} \;
```

5

```
# Comprimir todos los .txt en un tarball

$ tar czf backup.tgz *.txt && tar tf backup.tgz
```

6

```
# Listar PIDs de todos los bash en ejecución

$ ps aux | grep bash | grep -v grep | awk '{print $2}'
```

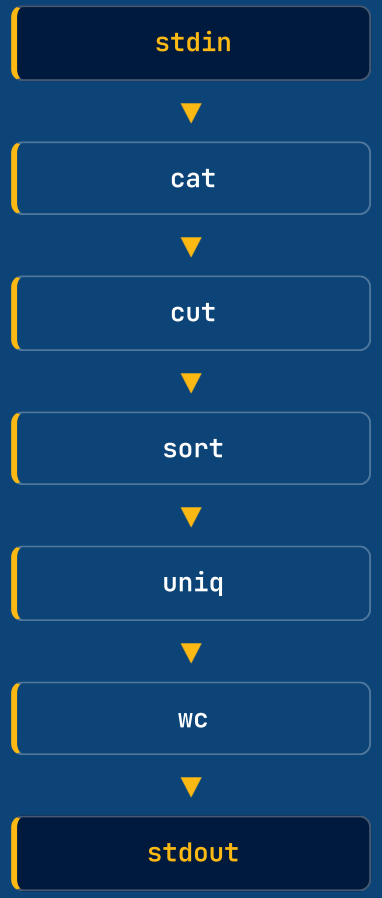
Variante sugerida: cambia el find del paso 4 por `find . -name "*.txt" | xargs wc -l` y compara la salida.

LPI

Pipeline de filtros y ciclo de vida de un proceso

Dos diagramas flat para fijar conceptos.

A) COMPOSICIÓN DE FILTROS



Cada filtro lee de stdin, escribe a stdout. La composición con | conecta la stdout de uno a la stdin del siguiente.

B) FORK · EXEC · EXIT · SIGCHLD



`wait()` permite al padre recoger el estado de salida del hijo y evitar procesos zombi.

Comprueba lo aprendido

Resuelve mentalmente; las soluciones están en la siguiente diapositiva.

Q1 ¿Qué imprime exactamente el comando `echo '$USER'` en bash si USER=alice?

Pista: revisa las reglas de quoting de la diapo 3.

Q2 Tienes un archivo con líneas duplicadas dispersas. Ejecutas `cat archivo | uniq | wc -l`. ¿Obtendrás las líneas únicas? ¿Por qué?

Pista: uniq sólo agrupa adyacentes.

Q3 ¿Qué hace exactamente la redirección `2>&1` en el comando `cmd > log 2>&1`?

Pista: el orden de las redirecciones importa.

Q4 Ejecutas `kill 1234` (sin número de señal). ¿Qué señal se envía por defecto y cómo se llama?

Pista: revisa la tabla de señales de la diapo 7.

Q5 En vi, quieres guardar los cambios y salir del editor con UN solo comando. ¿Cuál escribes?

Pista: hay tres formas válidas; nómbralas todas.

Soluciones del quiz

Compara tus respuestas con estas soluciones razonadas.

✓ **Q1** **Imprime literalmente la cadena `$USER` (no expande).**

Las comillas simples desactivan toda sustitución: variables, comandos y escape. Para expandir usa `"$USER"` (dobles) o `$USER` (sin comillas).

✓ **Q2** **NO. uniq solo agrupa líneas IDÉNTICAS ADYACENTES.**

Si las duplicadas están dispersas, no se eliminan. La receta correcta es `cat archivo | sort | uniq | wc -l`. sort reúne los duplicados y uniq los colapsa.

✓ **Q3** **Redirige stderr (FD 2) al mismo destino actual de stdout (FD 1).**

En `cmd > log 2>&1`, primero stdout apunta a log, luego stderr se fusiona con stdout (que ya es log). El orden importa: `2>&1 > log` haría algo distinto.

✓ **Q4** **SIGTERM (señal 15).**

Es la terminación elegante por defecto: el proceso puede atraparla, limpiar recursos y salir. Para forzar salida sin cleanup, usa `kill -9` (SIGKILL).

✓ **Q5** **`:wq` o `:x` o `ZZ` (en modo comando).**

Las tres guardan y salen. Diferencia fina: `:x` solo escribe si el buffer fue modificado (no actualiza mtime si no hubo cambios), mientras que `:wq` siempre escribe.

LPI

¡Sigue preparándote!

Has recorrido el tema con más peso del examen LPIC-1 101-500 v5.0. Practica los comandos en `/tmp/lpi103` y vuelve a este material cuando repases.

MATERIALES OFICIALES DE ESTUDIO

LPI Learning Materials

`https://learning.lpi.org/es/learning-materials/101-500/`

Capítulos gratuitos en español publicados por LPI para los temas 101, 102 y 103.

OBJETIVOS DEL EXAMEN

LPI Exam Objectives

`https://www.lpi.org/es/exam-101-102-objectives/`

Lista oficial de objetivos y pesos del examen 101-500 v5.0.

Material propio (no oficial) — no afiliado a LPI. Creado con fines docentes para preparar el examen LPIC-1 101-500 v5.0. Tema 103: Comandos GNU y Unix.